

Technical Requirements Document

Architecture, services, APIs, security, and deployment of the ElkaTech Launchpad platform

This TRD specifies the technical architecture of ElkaTech Launchpad: an npm-workspace monorepo containing a React/Vite frontend, a Fastify API gateway, four internal Fastify services (auth, catalog, service-desk, notification), a multi-schema PostgreSQL database, and a Vercel serverless deployment backed by Neon Postgres. It documents service boundaries, the gateway's security responsibilities, the authentication and authorization model, the request lifecycle, the outbox-based notification pipeline, the full API surface, and the operational requirements for deploying and running the platform.

Document	Technical Requirements Document (TRD)
Product	ElkaTech Launchpad
Version	1.0
Date	June 10, 2026
Status	Approved baseline — reflects the platform as built
Owner	ElkaTech Engineering
Audience	Backend, frontend, and platform engineers
Classification	Internal / Confidential
Source of truth	Repository README.md, migrations, and service source code

Contents

1. Technical Overview	4
2. Current Architecture	4
3. Monorepo Structure	6
4. Frontend Architecture	6
4.1 Stack and responsibilities	7
4.2 Key frontend modules	7
5. Backend Architecture	7
5.1 Fastify gateway (services/gateway, :4000)	8
5.2 Auth service (services/auth, :4001)	8
5.3 Catalog service (services/catalog, :4002)	8
5.4 Service-desk service (services/service-desk, :4003)	8
5.5 Notification service (services/notification, :4004)	8
6. API Gateway Responsibilities	9
7. Internal Service Communication	9
8. Authentication Architecture	10
8.1 Firebase ID token exchange	10
8.2 Cookie session and CSRF	10
8.3 Roles and approval status	11
9. Authorization Model	11
10. Request Lifecycle	11
10.1 Full transition matrix	12
10.2 Lifecycle events and rules	12
11. Notification Architecture	13
11.1 Event-to-email mapping	13
11.2 Delivery guarantees	14
12. Outbox Pattern	14
13. Deployment Architecture	14
13.1 Topology	15

13.2 Vercel specifics	15
13.3 Neon / PostgreSQL	15
13.4 Environment variables	16
14. API Endpoint Documentation	16
14.1 Auth endpoints	17
14.2 Catalog endpoints	17
14.3 Request endpoints	18
14.4 Admin endpoints	18
14.5 Health endpoints	18
15. Error Handling Requirements	19
16. Logging and Observability Requirements	19
17. Security Requirements	19
18. Performance Requirements	20
19. Testing Strategy	20
20. Migration Strategy	21
21. Production Deployment Checklist	21
22. Known Technical Risks	22
23. Future Technical Improvements	22
24. Assumptions and Open Questions	23

1. Technical Overview

ElkaTech Launchpad is a TypeScript monorepo. One React application serves both the public marketing site and the protected service portal; one Fastify gateway is the only browser-facing API; four internal Fastify services own their own PostgreSQL schemas; email flows run through transactional outbox tables polled by the notification service.

Concern	Choice
Language / runtime	TypeScript end-to-end on Node 22 LTS (pinned via .node-version)
Frontend	React 18, Vite, React Router, TanStack Query, Tailwind CSS, shadcn/Radix UI, Framer Motion
Backend	Fastify services with Zod validation
Identity	Firebase Authentication (email/password + Google); Firebase Admin SDK server-side; cookie sessions + CSRF + RBAC owned by the platform
Data	PostgreSQL — local Docker or Neon in production; four schemas (auth, catalog, service_desk, notification); optional Redis-compatible cache for catalog
Email	Nodemailer over SMTP (Mailpit locally); AWS SES templated sends for two production transactional flows
Tooling	npm workspaces, Vitest, ESLint
Deployment	Vercel: static Vite output + prebuilt serverless API bundles (api/*.mjs); Neon Postgres via Vercel integration

2. Current Architecture

The browser runs a client-side React application. All public API traffic goes to the gateway at /api/*, which owns browser-facing concerns — session cookies, CSRF checks, rate limiting, and role enforcement — then calls internal services with a shared x-internal-token header. Auth and service-desk actions write outbox events; the notification service polls those tables and converts them into email deliveries.

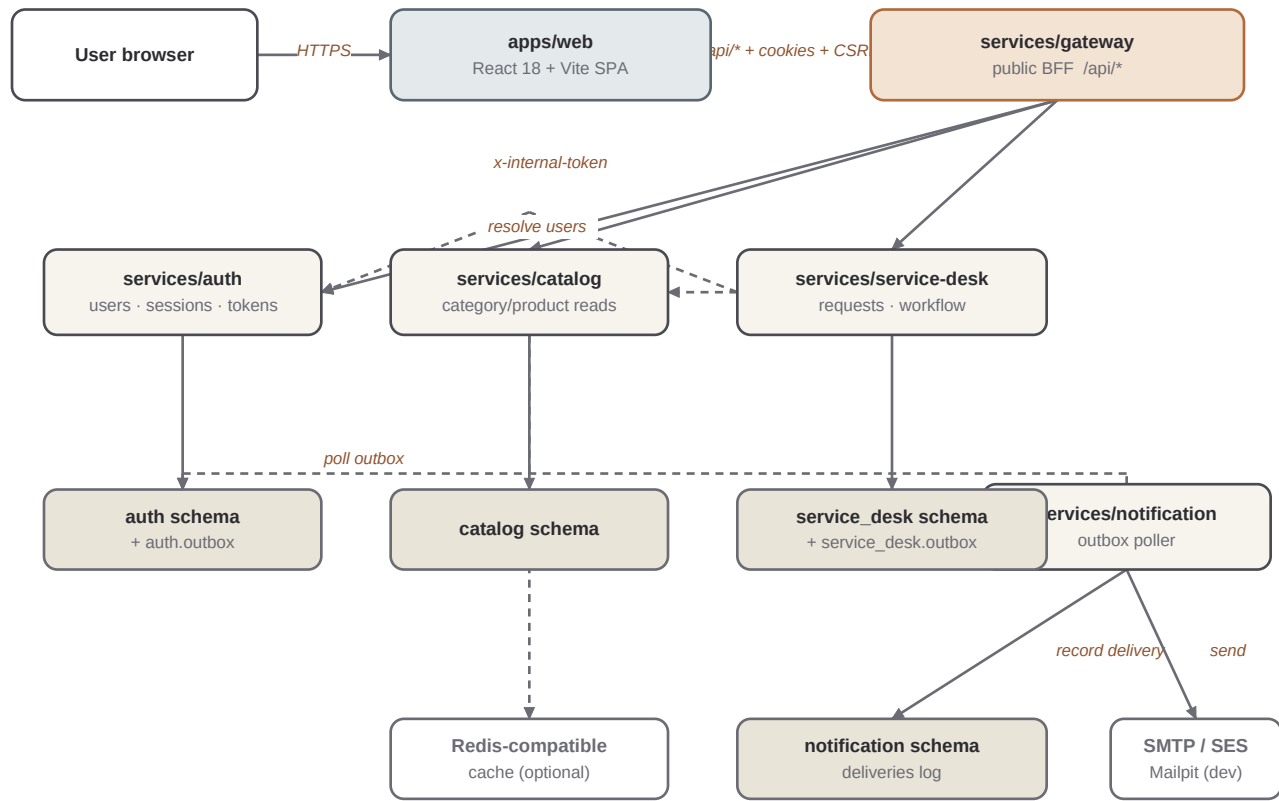


Figure 1 — System architecture. Solid arrows are synchronous calls; dashed arrows are optional or polling paths.

Equivalent Mermaid source (kept in the README and reusable in any Mermaid renderer):

```

flowchart TD
    Browser[User Browser] --> Web[apps/web React app]
    Web --> Gateway[services/gateway public API]
    Gateway --> Auth[services/auth]
    Gateway --> Catalog[services/catalog]
    Gateway --> Desk[services/service-desk]
    Desk --> Auth
    Desk --> Catalog
    Auth --> AuthSchema[(Postgres auth schema)]
    Catalog --> CatalogSchema[(Postgres catalog schema)]
    Desk --> DeskSchema[(Postgres service_desk schema)]
    Auth --> AuthOutbox[auth.outbox]
    Desk --> DeskOutbox[service_desk.outbox]
    Catalog -. optional cache .-> Redis[(Redis-compatible cache)]
    Notification[services/notification] --> AuthOutbox
    Notification --> DeskOutbox
    Notification --> NotificationSchema[(Postgres notification schema)]
    Notification --> SMTP[SMTP / Mailpit]
  
```

Listing 1 — Architecture diagram, Mermaid source.

3. Monorepo Structure

```
elkatech-launchpad/
|-- apps/
|   |-- web/                # React 18 + Vite SPA (public site + portal)
|   |   |-- public/         # Product images and static assets
|   |   |-- src/components/ # Landing, catalog, shell, and UI components
|   |   |-- src/hooks/      # Session and shared frontend hooks
|   |   |-- src/lib/         # API client and utility helpers
|   |   |-- src/pages/      # Auth and service-portal pages
|-- services/
|   |-- gateway/            # Public BFF/API boundary
|   |-- auth/               # Accounts, sessions, invitations, tokens
|   |-- catalog/            # Catalog read APIs and optional cache
|   |-- service-desk/       # Requests, messages, assignment, workflow
|   |-- notification/       # Outbox polling and email delivery
|-- packages/
|   |-- config/             # Env, DB, Redis, HTTP, internal-auth helpers
|   |-- contracts/          # Shared Zod schemas, types, catalog seed data
|-- scripts/
|   |-- db-migrate.ts       # Applies SQL migrations across services
|   |-- db-seed.ts          # Seeds catalog from shared contracts
|   |-- bootstrap-admin.ts  # Creates/updates the initial admin account
|   |-- cleanup-test-data.ts # Safe (dry-run-first) test data removal
|   |-- bundle-api.cjs       # Builds serverless handlers into api/*.mjs
|-- api/                    # Generated serverless handlers (committed)
|-- docker-compose.yml      # Local Postgres + Mailpit
|-- vercel.json             # Rewrites: /api/* -> bundles, SPA fallback
|-- package.json            # npm workspaces root
```

Listing 2 — Repository layout.

- npm workspaces tie the packages together; root build/test/lint fan out to workspace scripts.
- packages/contracts holds shared Zod schemas and types used by both frontend and services, plus the catalog seed data.
- packages/config centralizes env parsing (Zod), database and Redis clients, HTTP helpers (fetchJson with typed InternalFetchError), and internal-auth header helpers.
- Both package-lock.json and bun.lockb exist; the documented workflow is npm (root scripts and lockfile are npm-based).

4. Frontend Architecture

4.1 Stack and responsibilities

Technology	Role in apps/web
React 18	Component model for the public site and portal; a single app serves both.
TypeScript	End-to-end typing; shared types imported from packages/contracts.
Vite	Dev server (127.0.0.1:8080, proxying /api to the gateway on :4000) and production build to apps/web/dist; reads env from the workspace root via envDir so VITE_FIREBASE_* lives in the same .env as the backend.
React Router	BrowserRouter with client-side routes: public landing/category routes, auth pages, and protected /app/* portal routes; /inject-printer is a Navigate redirect.
TanStack Query	Server-state management. The ['session'] query backs ProtectedRoute via GET /api/auth/me; login/signup set the session cache synchronously (setQueryData) before navigating, then invalidate for background freshness — this fixed the Google sign-in first-click race.
Tailwind CSS	Utility styling over CSS custom properties; design tokens in src/index.css and tailwind.config.ts; portal tokens: lp-card, --lp-panel, --lp-accent, --lp-line.
shadcn/Radix UI	Accessible primitives (dialogs, selects, toasts, tooltips) composed into the component library.
Framer Motion	Controlled animation: expressive in the hero and intro loader; everywhere else the StableReveal primitive limits entrances to opacity/blur so layout never shifts; prefers-reduced-motion is respected.

4.2 Key frontend modules

Module	Responsibility
src/lib/api.ts	Single API client: all requests include credentials; mutating requests attach x-csrf-token read from the CSRF cookie.
ProtectedRoute	Session gate with role-based fallback for /app routes.
PortalShell	Collapsible sidebar shell: navigation, theme selector, profile block (display name + email), My Account entry, logout.
ThemeProvider / ThemeToggle	light/dark/system with persistence under localStorage key elkatech-ui-theme; updates the documentElement class.
src/lib/request-status.ts	Frontend mirror of the workflow transition map; getAllowedStatuses filters actions to valid, role-appropriate targets.
ProductCategoryPage	Shared category layout: carousel, specs, CTAs, theme-aware background.

5. Backend Architecture

Five Fastify services. In local development each runs as its own HTTP process on ports 4000–4004; on Vercel the same entry points are bundled into serverless handlers. All request/response payloads are validated with Zod schemas from packages/contracts.

5.1 Fastify gateway (services/gateway, :4000)

The only browser-facing service. Terminates sessions and CSRF, performs role and approval checks, rate-limits sensitive routes, validates input, and forwards to internal services with the shared internal token. It also aggregates health probes for the admin dashboard. Internal errors surface with their real status codes: fetchJson throws a typed InternalFetchError {status, body} that the gateway forwards instead of collapsing everything to 500.

5.2 Auth service (services/auth, :4001)

- Owns auth.users, auth.sessions, auth.tokens, auth.oauth_identities, and auth.outbox.
- Password auth (legacy), Firebase ID-token verification via the Firebase Admin SDK, Google OAuth identity linking, session issue/lookup, verify/reset/invite tokens.
- Approval workflow state on the user row (pending_approval / approved / rejected / suspended) plus account origin (self_signup / admin_invite / firebase_google / legacy).
- Emits outbox events: user.registered, user.email_verified, user.password_reset_requested, user.approval_changed.

5.3 Catalog service (services/catalog, :4002)

- Read-only API over catalog.categories and catalog.products.
- Data is seeded from catalogSeedData in packages/contracts via npm run db:seed.
- Optional Redis-compatible cache (KV_URL / REDIS_URL) in front of reads.

5.4 Service-desk service (services/service-desk, :4003)

- Owns service_desk.requests, request_messages, request_history, and service_desk.outbox.
- Request lifecycle: creation with product snapshot (fetched from catalog), strict status workflow (workflow.ts), claim/assign/reassign rules, role-aware visibility.
- Resolves participant identities (assignee, message authors) through the auth service so request detail can show real names to every role.
- Emits outbox events: request.created, request.assigned, request.status_changed, request.customer_message_posted, request.staff_reply_posted.

5.5 Notification service (services/notification, :4004)

- Polls auth.outbox and service_desk.outbox for unprocessed events.
- Builds emails per event type; sends via AWS SES templates for the two formalized transactional flows (account added on admin invite; request claimed/assigned to customer + engineer) and via Nodemailer/SMTP for everything else and for local Mailpit.
- Records one row per (event, recipient) in notification.deliveries with sent/failed status, attempt count, and last error; a failure for one recipient does not block the others.

- On Vercel, the outbox is also drained inline after mutating operations because long-lived background pollers do not exist in serverless functions.

6. API Gateway Responsibilities

Responsibility	Behavior
Sessions	Issues and validates the session cookie (elkatech_session by default). Sessions are server-side rows in auth.sessions storing a hashed token, user agent, IP, expiry (SESSION_TTL_HOURS; 168h in the example env), and last-seen time. /api/auth/me resolves the current user including approvalStatus.
CSRF	A CSRF cookie (elkatech_csrf) pairs with the session; every mutating request must echo it in the x-csrf-token header. assertCsrf rejects mismatches.
Rate limiting	Sensitive endpoints are rate-limited (e.g. POST /api/me/password-reset), which also returns a constant response so account existence cannot be probed.
RBAC	requireSession plus role guards: engineer/admin for queue, claim, and status routes; admin for /api/admin/* and assignment. Approval gating on request creation returns USER_PENDING_APPROVAL / USER_REJECTED / USER_SUSPENDED.
Request validation	Zod input schemas from packages/contracts (signInInputSchema, createServiceRequestInputSchema, etc.) validate every body before forwarding.
Internal routing	Forwards to AUTH_SERVICE_URL, CATALOG_SERVICE_URL, SERVICE_DESK_URL, NOTIFICATION_SERVICE_URL with the x-internal-token header; forwards real downstream status codes via InternalFetchError.
Health aggregation	GET /api/admin/health probes every internal service's GET /health and returns healthy / degraded / down per service with latency.

7. Internal Service Communication

- Transport is plain HTTP between services: localhost ports in development; on Vercel, the deployment calls itself on its own URL (VERCEL_URL for previews, VERCEL_PROJECT_PRODUCTION_URL for production) — each deployment is self-contained end-to-end.
- Every non-health internal endpoint requires the shared secret header x-internal-token (INTERNAL_SERVICE_TOKEN); ensureInternal rejects calls without it. Set a strong random value in production.
- The shared fetchJson helper (packages/config/src/http.ts) attaches internal headers, strips the JSON content type on body-less requests (a DELETE bug fix), and throws typed InternalFetchError {status, body} so upstream services can forward real errors.
- Service-desk calls auth (resolve users, list engineers) and catalog (product snapshot at request creation).
- Asynchronous communication uses the outbox tables — there is no message broker.

Operational. Vercel Deployment Protection must be disabled for previews (or set to 'Only Production Deployments'): the platform's server-to-server self-calls cannot pass Vercel's SSO wall, so Standard Protection on every deployment breaks internal routing.

8. Authentication Architecture

Firebase Authentication is the identity provider for new accounts; the platform keeps its own cookie-session + CSRF + RBAC model. Firebase proves who the user is; the local auth schema decides what they may do.

8.1 Firebase ID token exchange

1. Frontend signs in with the Firebase Web SDK — `createUserWithEmailAndPassword` (+ `sendEmailVerification`), `signInWithEmailAndPassword`, or `signInWithPopup` with the Google provider.
2. Frontend obtains the Firebase ID token and POSTs it to `/api/auth/firebase/session` as `{ idToken }`.
3. Gateway/auth verify the token with the Firebase Admin SDK (`FIREBASE_PROJECT_ID / FIREBASE_CLIENT_EMAIL / FIREBASE_PRIVATE_KEY`).
4. Auth service creates or syncs the local user row (`firebase_uid`, email, display name, email_verified; new self-signups get role customer and approval_status pending_approval).
5. A session row is created; the session cookie and CSRF cookie are set; the frontend seeds the ['session'] query and navigates into the portal.

```
sequenceDiagram
    participant FE as Web app
    participant FB as Firebase Auth
    participant GW as Gateway
    participant AU as Auth service
    FE->>FB: signInWithPopup / signInWithEmailAndPassword
    FB-->>FE: Firebase ID token
    FE->>GW: POST /api/auth/firebase/session { idToken }
    GW->>AU: verify token (Firebase Admin SDK)
    AU->>AU: create/sync auth.users (role, approval_status)
    AU-->>GW: user + session
    GW-->>FE: Set-Cookie: elkatech_session, elkatech_csrf
    FE->>GW: GET /api/auth/me (ProtectedRoute)
    GW-->>FE: { user incl. approvalStatus }
```

Listing 3 — Firebase session bridge, Mermaid sequence.

8.2 Cookie session and CSRF

- Session cookie name `elkatech_session`; CSRF cookie `elkatech_csrf` (both configurable). Session lifetime via `SESSION_TTL_HOURS` (168h in `.env.example`; schema default 720h).
- Session tokens are stored hashed (`token_hash`) with a per-session `csrf_token`, user agent, IP, expiry, and last-seen tracking.
- Mutations require the `x-csrf-token` header to match the CSRF cookie.
- Logout deletes the session; user removal and suspension delete all of a user's sessions.

8.3 Roles and approval status

- Role is fixed per account: customer (self-signup), engineer or admin (admin-invited). Account origin records how the account was created (self_signup, admin_invite, firebase_google, legacy); self-signups are locked to the customer role.
- Approval status (pending_approval / approved / rejected / suspended) is independent of email verification and is enforced at the gateway on request creation, plus surfaced via /api/auth/me to drive frontend state cards.
- Legacy flows remain: legacy signup/login/verify/reset endpoints and the server-side Google OAuth flow (/api/auth/google/start|callback) with identity linking in auth.oauth_identities, requesting only openid email profile scopes.

9. Authorization Model

Action	customer	engineer	admin
GET /api/requests (own scope)	Yes	Yes	Yes
GET /api/requests?scope=queue	No	Yes	Yes
POST /api/requests	Yes — requires verified email + approval_status approved	No	No
GET /api/requests/:id	Own requests only	Visible scope	All
POST .../messages (customer_visible)	Own requests	Yes	Yes
POST .../messages (internal_note)	No	Yes	Yes
POST .../claim	No	Yes	Yes
POST .../assign	No	No	Yes
POST .../status (forward transitions)	No	Yes	Yes
POST .../status (reopen resolved/closed → new)	No	No	Yes
GET/POST /api/admin/*	No	No	Yes
PATCH /api/me/profile, POST /api/me/password-reset	Yes	Yes	Yes

Authorization is layered: the gateway enforces session + CSRF + role + approval; the service-desk re-checks workflow legality (canTransitionRequestTo combines RBAC, the transition map, and the admin-only reopen gate in a single check); internal endpoints additionally require the internal token. The frontend mirrors the same rules to hide actions it knows will fail.

10. Request Lifecycle

Requests move through a strict state machine enforced in services/service-desk/src/workflow.ts and mirrored in the frontend (src/lib/request-status.ts). 'assigned' is never a /status target — assignment is set exclusively by /claim and /assign. Same-status updates are rejected so history never accumulates no-op rows.

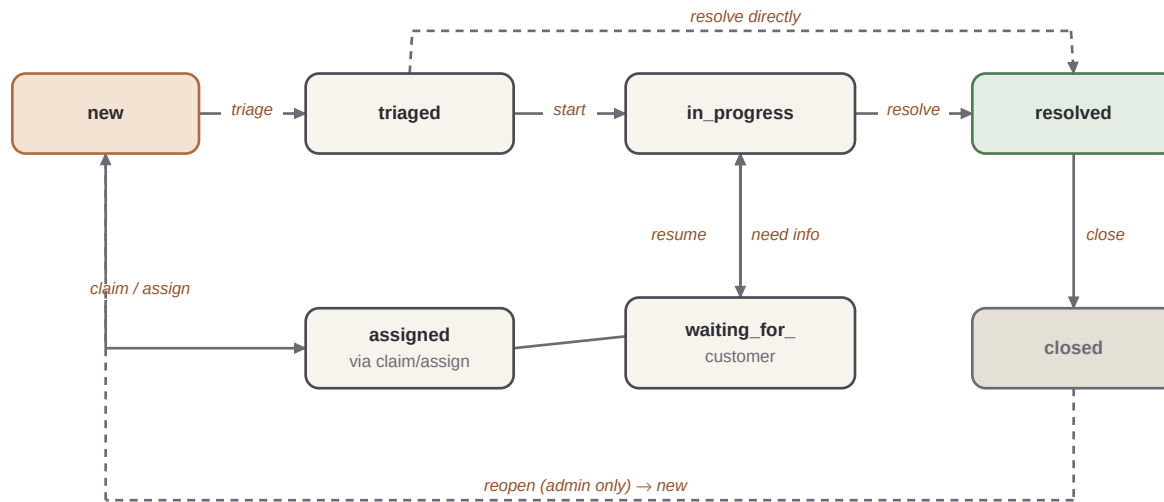


Figure 2 — Request workflow. Dashed edges are secondary paths; closed/resolved → new is admin-only. Most states may also move directly to closed (see matrix).

10.1 Full transition matrix

From	Allowed /status targets
new	triaged, closed
triaged	in_progress, waiting_for_customer, resolved, closed
assigned	triaged, in_progress, waiting_for_customer, resolved, closed
in_progress	waiting_for_customer, resolved, closed
waiting_for_customer	in_progress, resolved, closed
resolved	new (admin only), closed
closed	new (admin only)

10.2 Lifecycle events and rules

- **Creation:** gateway validates the customer (session, CSRF, verified email, approved), service-desk snapshots the product from catalog, stores the request with a unique request_number and status new, writes request_created history, and emits request.created to the outbox.
- **Claim:** 409 with a friendly message if already assigned. **Assign:** no-ops when reassigning the same engineer. Reassignment writes a distinct request_reassigned history event with {previousEngineerId, engineerId, engineerEmail} metadata.

- Status change: canTransitionRequestTo returns 400 'Invalid workflow transition.' or 403 'Forbidden'; the gateway forwards the real status code.
- History event types include: request_created, request_claimed, request_assigned, request_reassigned, status_changed, request_updated, request_archived, request_cancelled.

11. Notification Architecture

Notifications are asynchronous and database-driven. State-changing operations write an event row to their service's outbox table inside the same transaction as the business change; the notification service turns events into emails and records the outcome.

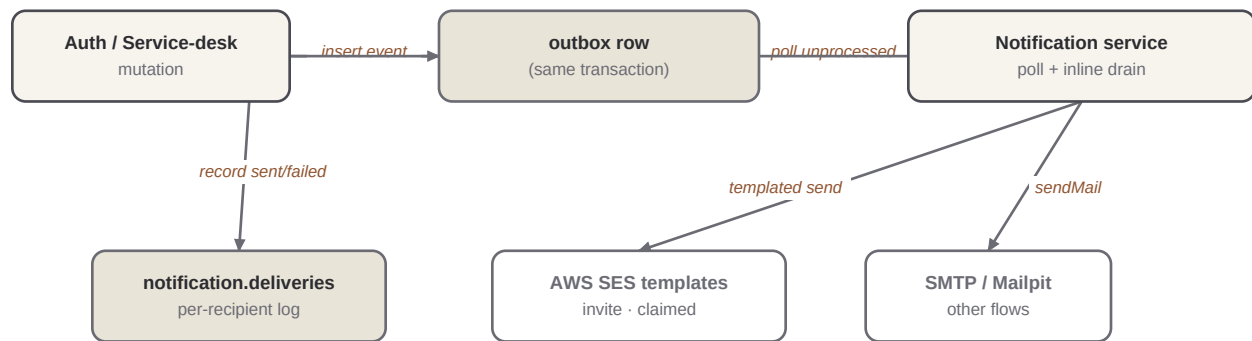


Figure 3 — Outbox-driven notification pipeline.

11.1 Event-to-email mapping

Outbox event	Recipients	Transport
user.registered (with invitation payload)	Invited staff member	SES template ElkaTechAccountAdded
user.registered (self-signup)	New user (verification handled by Firebase for Firebase accounts; legacy verify email otherwise)	SMTP
user.email_verified, user.password_reset_requested, user.approval_changed	Affected user	SMTP
request.created	Customer confirmation	SMTP
request.assigned (claim or assign)	Customer and engineer (deduplicated)	SES template ElkaTechRequestClaimed
request.status_changed, request.customer_message_posted, request.staff_reply_posted	Counterparty (customer or staff)	SMTP

Precision. Exact recipient lists for the SMTP flows follow the notification service's buildEmails mapping; the table reflects event names confirmed in the codebase. When SES is not configured, the SES flows fall back to SMTP so local development works unchanged with Mailpit.

11.2 Delivery guarantees

- At-least-once: events stay unprocessed until handled; deliveries are idempotent per (event_id, recipient_email) unique index, so retries cannot double-record.
- Per-recipient isolation: the outbox drains per recipient, so one failing address (e.g. an SES sandbox-unverified recipient) does not block other recipients of the same event.
- Failures are recorded with status failed, attempt count, and last_error for diagnosis.
- Serverless reality: on Vercel the outbox is drained inline after mutations (no resident poller); locally the service polls continuously.

12. Outbox Pattern

The transactional outbox is the platform's only asynchronous mechanism, chosen over a message broker for operational simplicity:

1. A business transaction (e.g. create request) writes its rows and an event row into the same schema-local outbox table — atomically.
 2. Outbox rows carry id, aggregate_type, aggregate_id, event_type, jsonb payload, occurred_at, and processed_at (null until handled).
 3. The notification service selects unprocessed rows, performs the side effect (email), records the delivery, and marks processed_at.
 4. If the process crashes mid-way, the unprocessed event is picked up again; the per-recipient unique index keeps the retry idempotent.
- Two outboxes exist (auth.outbox, service_desk.outbox), keeping each service's writes inside its own schema boundary.
 - Events are facts, not commands: payloads carry the data needed to build emails (emails, names, request numbers, URLs) so the notification service rarely needs to call back into services.

13. Deployment Architecture

13.1 Topology

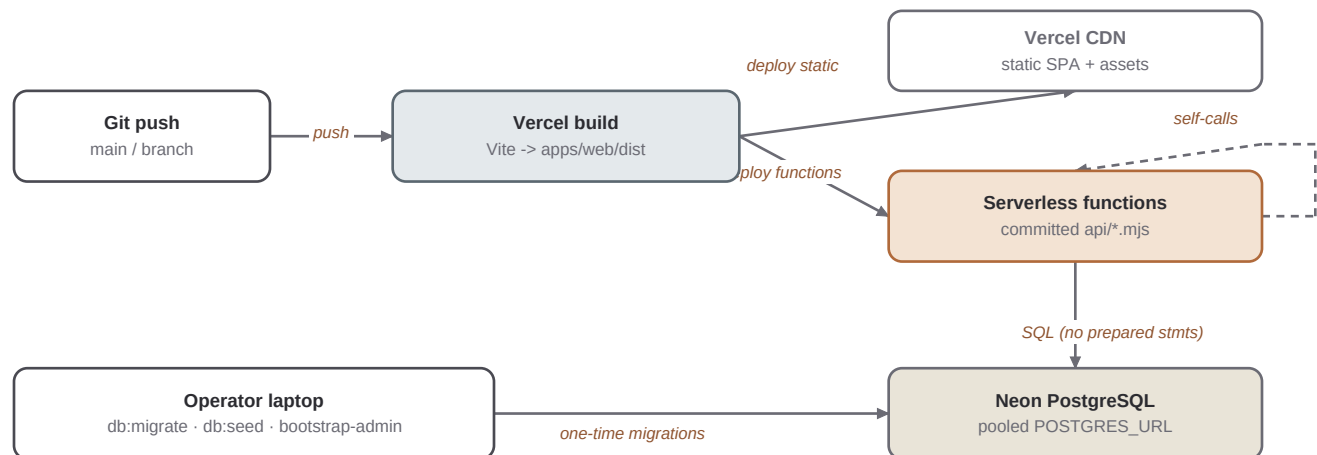


Figure 4 — Vercel + Neon deployment. Internal service calls loop back to the same deployment's URL.

13.2 Vercel specifics

- Root vercel.json rewrites /api/* to bundled handlers in api/*.mjs and all remaining routes to index.html (SPA fallback). Output directory: apps/web/dist; Framework preset: Vite.
- scripts/bundle-api.cjs builds the serverless handlers from service entry points. The bundles are committed; Vercel runs them as-is and does not rebundle — after any change under services/* or packages/config, regenerate and commit (node scripts/bundle-api.cjs && git add api/*.mjs).
- packages/config/src/env.ts rewrites service URLs when VERCEL=1: previews talk to their own VERCEL_URL, production to VERCEL_PROJECT_PRODUCTION_URL — each deployment is self-contained.
- env.ts walks up the directory tree to the workspace-root .env so every service loads the same file locally; Vite reads the same root .env via envDir.
- Pushes to main deploy production automatically; feature branches get preview URLs sharing the production database (Neon branch-per-deployment is deliberately off).

13.3 Neon / PostgreSQL

- Neon is the recommended hosted Postgres; connect through Vercel's Storage integration with the custom env prefix POSTGRES (the code reads POSTGRES_URL, falling back to DATABASE_URL locally).
- Disable 'Create Database Branch For Deployment' for both Preview and Production, otherwise every deployment gets an empty branch and migrations only land on main.
- Prepared statements are disabled on the pgbouncer pooled connection to avoid stale-plan errors.
- Migrations/seed/bootstrap are run once from an operator machine against the pooled URL (quoted in single quotes — it contains ampersands).

13.4 Environment variables

packages/config/src/env.ts (Zod schema) is the source of truth; .env.example is the local template. Key groups:

Group	Variables	Notes
Core	NODE_ENV, POSTGRES_URL, DATABASE_URL, KV_URL, REDIS_URL	POSTGRES_URL preferred in production; local default postgres://elkatech:elkatech@127.0.0.1:5432/elkatech
Internal mesh	INTERNAL_SERVICE_TOKEN, GATEWAY_URL, AUTH_SERVICE_URL, CATALOG_SERVICE_URL, SERVICE_DESK_URL, NOTIFICATION_SERVICE_URL, APP_BASE_URL	Token must be strong random in production; URLs auto-rewritten on Vercel
Sessions	SESSION_COOKIE_NAME, CSRF_COOKIE_NAME, SESSION_TTL_HOURS	Defaults elkatech_session / elkatech_csrf; 168h in example env
Email (SMTP)	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM	Port 465 implies TLS; FROM accepts display-name form; Mailpit locally (1025)
Email (SES)	AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, SES_FROM_EMAIL, SES_ACCOUNT_ADDED_TEMPLATE, SES_REQUEST_CLAIMED_TEMPLATE	Used for the two templated transactional flows in production
Firebase (server)	FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY	Admin SDK service account; escaped \n allowed in the key; never log or commit
Firebase (web)	VITE_FIREBASE_API_KEY, _AUTH_DOMAIN, _PROJECT_ID, _APP_ID, _STORAGE_BUCKET, _MESSAGING_SENDER_ID	Public client identifiers — not secrets
Legacy OAuth	GOOGLE_OAUTH_CLIENT_ID, _CLIENT_SECRET, _REDIRECT_URI	Optional server-side Google flow
Bootstrap	BOOTSTRAP_ADMIN_EMAIL, BOOTSTRAP_ADMIN_PASSWORD	Initial admin; also provisioned in Firebase when Admin SDK is configured
Platform	VERCEL, VERCEL_URL, VERCEL_PROJECT_PRODUCTION_URL	Provided by Vercel; drive URL rewriting

14. API Endpoint Documentation

All endpoints are served by the gateway under /api. Mutating requests require the session cookie plus x-csrf-token. 'Legacy' marks pre-Firebase flows kept for existing accounts.

14.1 Auth endpoints

Method	Endpoint	Purpose	Auth
POST	/api/auth/signup	Create customer account or accept invite (legacy + invite path); body signUpInputSchema	Public
POST	/api/auth/login	Create session and cookies (legacy); body loginInputSchema	Public
POST	/api/auth/logout	End session	Session if present
GET	/api/auth/me	Resolve current session; returns { user } incl. approvalStatus	Public
POST	/api/auth/forgot-password	Request reset email (legacy)	Public
POST	/api/auth/reset-password	Complete reset (legacy)	Public
POST	/api/auth/verify-email	Verify email token (legacy)	Public
POST	/api/auth/firebase/session	Verify Firebase ID token and create local session; body { idToken }	Public
GET	/api/auth/google/start	Start legacy server-side Google OAuth (?returnTo=&inviteToken=)	Public
GET	/api/auth/google/callback	Legacy Google OAuth callback; redirects after auth	Public
PATCH	/api/me/profile	Update own display name only; strict schema { displayName: string(2..80) }	Session + CSRF
POST	/api/me/password-reset	Send reset email to own address; rate-limited, constant response	Session + CSRF

14.2 Catalog endpoints

Method	Endpoint	Purpose	Auth
GET	/api/catalog/categories	List catalog categories	Public via gateway
GET	/api/catalog/products	List products; optional ?category= filter	Public via gateway
GET	/api/catalog/products/:productId	Read one product	Public via gateway

14.3 Request endpoints

Method	Endpoint	Purpose	Auth
POST	/api/requests	Create service request; body createServiceRequestInputSchema	Customer with verified email + approved status
GET	/api/requests	List visible requests; ?scope=queue for staff queue	Authenticated (role-aware)
GET	/api/requests/:requestId	Request with messages, history, assignedEngineer, author names	Authenticated with request access
POST	/api/requests/:requestId/messages	Add message or internal note; body createRequestMessageInputSchema	Authenticated (notes: staff only)
POST	/api/requests/:requestId/claim	Claim an unassigned request; 409 if already assigned	Engineer / admin
POST	/api/requests/:requestId/assign	Assign or reassign an engineer; body { engineerId }	Admin
POST	/api/requests/:requestId/status	Change status per transition map; body updateRequestStatusInputSchema	Engineer / admin (reopen: admin)

14.4 Admin endpoints

Method	Endpoint	Purpose	Auth
GET	/api/admin/users	List users incl. approvalStatus	Admin
GET	/api/admin/users/summary	Approval-status counts for the dashboard	Admin
POST	/api/admin/users/invite	Invite engineer/admin; body inviteUserInputSchema	Admin
POST	/api/admin/users/:userId/approve	Approve pending customer	Admin
POST	/api/admin/users/:userId/reject	Reject pending customer	Admin
POST	/api/admin/users/:userId/suspend	Suspend approved customer	Admin
POST	/api/admin/users/:userId/reactivate	Reactivate suspended/rejected customer	Admin
DELETE	/api/admin/users/:userId	Permanently remove a non-admin user and their request data (cross-schema transaction; best-effort Firebase deletion)	Admin

14.5 Health endpoints

Method	Endpoint	Purpose	Auth
GET	/health (each service)	Heartbeat: { ok, service, environment }	None (internal)
GET	/api/admin/health	Probe all internal services; healthy / degraded / down + latency per service	Admin

15. Error Handling Requirements

- Validation errors: Zod schema failures return 400 with details; the frontend surfaces friendly messages.
- Authorization errors: 401 for missing/expired sessions; 403 for role/CSRF failures; approval gating returns the specific codes `USER_PENDING_APPROVAL`, `USER_REJECTED`, `USER_SUSPENDED` so the UI can render the right state card.
- Workflow errors: 400 'Invalid workflow transition.' for illegal status moves; 403 'Forbidden' for role-blocked moves; 409 for claiming an already-assigned request.
- Internal call errors: `fetchJson` throws `InternalFetchError {status, body}`; the gateway forwards the downstream status instead of collapsing to 500. Body-less requests must not send a JSON content type (historical `FST_ERR_CTP_EMPTY_JSON_BODY` fix).
- Email errors: per-recipient failures are recorded (`status='failed'`, `last_error`) without aborting other recipients; outbox rows remain available for retry semantics.
- Frontend: TanStack Query drives retry/stale behavior; toasts surface mutation failures; blocked states render as designed cards, not raw errors.

16. Logging and Observability Requirements

- Services use Fastify's structured logger (`app.log`) for request and error logging.
- Never log secrets: Firebase private key, ID tokens, session cookies, and CSRF tokens are explicitly excluded from logs.
- Health: every service exposes `GET /health`; the gateway aggregates healthy/degraded/down with latency; the admin dashboard refreshes the snapshot every 60 seconds. There is intentionally no external monitoring dependency.
- `notification.deliveries` doubles as an email audit log (event, recipient, subject, status, attempts, last error, timestamps).
- `service_desk.request_history` is the per-request audit trail (actor, role, event type, metadata, timestamp).

Assumption / open question. Production observability beyond heartbeats (log aggregation, error tracking, alerting, backups) is documented as a future improvement and is not yet formalized.

17. Security Requirements

Area	Requirement
Transport	All production traffic over HTTPS (Vercel TLS).
Sessions	Server-side sessions with hashed tokens; cookie lifetime bounded by <code>SESSION_TTL_HOURS</code> ; sessions deleted on logout, suspension, and account removal.

Area	Requirement
CSRF	Double-submit cookie pattern: x-csrf-token header must match the CSRF cookie on every mutation.
RBAC	Role checks at the gateway; admin-only surface for user management, assignment, reopen, and health.
Approval gating	Customers cannot create requests until admin-approved; specific error codes per blocked state.
Internal mesh	x-internal-token (strong random in production) required on all non-health internal endpoints.
Identity	Firebase ID tokens verified server-side with the Admin SDK; Google OAuth limited to openid email profile scopes; OAuth identities linked in a dedicated table.
Secrets	Service-account JSON and private keys are gitignored; FIREBASE_PRIVATE_KEY supports escaped newlines; secrets never logged. VITE_FIREBASE_* values are public client identifiers, not secrets.
Input validation	Zod schemas on every gateway input; strict minimal schemas for sensitive routes (e.g. profile update accepts only displayName 2..80).
Abuse resistance	Rate limiting on sensitive routes; constant responses on password-reset to prevent account enumeration.
Data deletion	User removal is a cross-schema transaction with admin accounts protected; Firebase deletion is best-effort post-commit.

18. Performance Requirements

- Static frontend served from Vercel's CDN; product images are static assets under apps/web/public/images.
- Serverless API: cold-start sensitive; bundles are prebuilt (api/*.mjs) to keep functions lean.
- Database access through Neon's pooled connection string; prepared statements disabled on the pgbouncer pooler to avoid stale-plan errors.
- Catalog reads may be cached in a Redis-compatible store (KV_URL / REDIS_URL) — optional, off by default locally.
- Request-detail enrichment resolves participants in parallel (Promise-level fan-out) to keep one round trip per user lookup.
- Frontend: TanStack Query caching avoids redundant fetches; the admin dashboard polls on a deliberate 60-second cadence; animations avoid layout-shifting properties.

Assumption / open question. No numeric latency/throughput SLOs are defined yet. Recommended starting point once measured: p95 < 500 ms for gateway reads, p95 < 1.5 s for request creation including snapshot fetch.

19. Testing Strategy

- Unit/integration tests run with Vitest via npm run test (root fans out to workspaces that define a test script).
- Current automated coverage focuses on the request workflow rules (transition map / RBAC gating) — the highest-risk pure logic.
- Frontend tests run via the web workspace (npm run test -w @elkatech/web).

- Linting via ESLint; note the documented pre-existing lint failures in shared UI/config files — changes should not add new ones, and the baseline cleanup is planned work.
- Type safety is enforced with `tsc --noEmit` against `apps/web/tsconfig.json` and workspace builds.
- Manual smoke flow for releases: `signup` → `pending card` → `admin approve` → `create request` → `claim` → `status walk` → emails in Mailpit/delivery log (see Production Deployment Checklist).

Recommended expansion (documented future improvement): broaden automated coverage beyond workflow tests — gateway auth/CSRF guards, approval gating, notification event mapping, and an end-to-end request lifecycle test against a disposable database.

20. Migration Strategy

- SQL migrations live per service under `services/<name>/migrations` and are applied in order by `scripts/db-migrate.ts` (`npm run db:migrate`).
- Migrations are written to be idempotent (create table/index if not exists; guarded constraint additions) so re-runs are safe.
- Existing history: `auth` 001 init → 002 `oauth_identities` → 003 `firebase_uid` + approval workflow (backfills existing users to approved) → 004 `account_origin` (backfills staff to `admin_invite`, old customers to legacy) → 005 `removed_at/removed_by` soft-delete markers. `Catalog`, `service_desk`, and `notification` each have an init migration; `notification` 002 relaxes delivery uniqueness to (`event_id`, `recipient_email`).
- Production migrations are run manually from an operator machine against the Neon pooled URL, followed by `db:seed` and `db:bootstrap-admin` as needed; previews share that database, so preview-only schema drift must be avoided.
- Code is written schema-tolerant where rollout order matters: the `auth` service probes `information_schema` for optional columns (e.g. `removed_at`) and degrades gracefully when a migration has not landed yet.

21. Production Deployment Checklist

1. Connect the GitHub repo to a Vercel project (Framework preset: Vite; build command and output directory from `vercel.json`).
2. Set Production + Preview env vars: `VITE_FIREBASE_*` (web), `FIREBASE_PROJECT_ID` / `FIREBASE_CLIENT_EMAIL` / `FIREBASE_PRIVATE_KEY` (Admin SDK), `INTERNAL_SERVICE_TOKEN` (strong random), `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS` / `SMTP_FROM`, SES settings, `BOOTSTRAP_ADMIN_EMAIL` / `BOOTSTRAP_ADMIN_PASSWORD`.
3. Add the production domain (and preview pattern if needed) to Firebase Authorized Domains.
4. Connect Neon via Vercel Storage with env prefix `POSTGRES`; disable branch-per-deployment for both Preview and Production; confirm `POSTGRES_URL` exists in project env.
5. Run migrations, seed, and bootstrap-admin once against the Neon pooled URL (single-quoted — it contains ampersands).

6. Set Vercel Deployment Protection to 'Only Production Deployments' or off, so internal self-calls work on previews.
7. Regenerate and commit serverless bundles after any backend change: `node scripts/bundle-api.cjs`; `git add api/*.mjs`.
8. Redeploy with build cache off so functions pick up new env vars.
9. Verify: admin health dashboard all-healthy; full request lifecycle smoke test; notification deliveries recorded.

22. Known Technical Risks

Risk	Details	Mitigation
Stale serverless bundles	Vercel runs committed <code>api/*.mjs</code> ; forgetting to rebundle ships old backend code	Documented step; automate in CI (future improvement)
Dual catalog sources	Public component arrays vs. seeded DB catalog drift	Update procedure touches both; unify generation later
Shared prod DB for previews	Preview deployments mutate production data	Deliberate (migration sanity); restrict preview testing; consider masked staging data
Manual migrations	Laptop-run migrations can be skipped or mis-ordered	Idempotent SQL; schema-tolerant code paths; future CI gate
Internal token is a single shared secret	Compromise grants internal API access	Strong random value; rotate periodically; never log
Serverless outbox draining	No resident poller on Vercel; inline drain ties email latency to request path	Keep sends fast; consider scheduled drains (cron) later
SES sandbox behavior	Unverified recipients fail individually	Per-recipient drain + failure logging; move SES out of sandbox
Node version drift	Old local Node breaks <code>tsx/concurrently</code>	<code>.node-version</code> pinned to 22; <code>nvm</code> alias default 22
Lint baseline	Pre-existing failures mask new issues	Planned cleanup; isolate new findings in review

23. Future Technical Improvements

- Automate API bundling in package scripts or CI so committed bundles can never go stale.
- Reconcile or generate the duplicate catalog datasets from one source.
- Expand automated coverage beyond workflow tests (gateway guards, notification mapping, end-to-end lifecycle).
- Formalize observability: log aggregation, error tracking, alerting, and database backup/restore procedures.
- Clean the lint baseline and enforce it in CI.
- Consider a scheduled (cron) outbox drain on Vercel to decouple email latency from request handling.

- Evaluate consolidating email delivery on one provider (SES templates vs. SMTP relay) once volumes justify it.

24. Assumptions and Open Questions

- **SLOs.** Latency/uptime targets are proposals; measure first, then commit.
- **Rate-limit coverage.** The README confirms rate limiting as a gateway concern and on `/api/me/password-reset` specifically; the full per-route limit table should be documented from gateway source.
- **SMTP recipient matrix.** Event names are confirmed; the exact recipient/subject matrix for each SMTP flow follows the notification service's `buildEmails` implementation and should be appended here verbatim when finalized.
- **Redis usage.** The catalog cache is optional and unset by default; confirm whether production enables it.
- **Backups.** Neon-side backup/restore procedures are not yet documented as operational runbooks.
- **Removed-user semantics.** Schema retains `removed_at`/`removed_by` soft-delete markers while the current admin remove action performs a permanent hard delete; decide whether the soft-delete columns remain as a safety net or are retired.